UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

Facultad de Ciencias Químicas e Ingeniería

Práctica 5

Materia:
Organización y Arquitectura de Computadoras

Docente:
Garcia Rocha Jose Isabel

Elaborado por:
Liang Liu Kevin

Tijuana, B.C. a 25 de marzo de 2026

Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Desarrollo

1. (25%) Implementar una subrutina la cual debe buscar, contar y mostrar en qué posición de una cadena de texto aparece una letra ingresada por el usuario, recibe en edx la dirección de la cadena a utilizar, y dentro de la subrutina debe pedir la letra a buscar.

```
C/C++
section .data
    msj: db 'Texto: ',0x0
    msj1: db 'La letra se encontro en posicion #',0x0
    msj2: db 'La letra se encontro # veces',0x0

section .bss
    cad1: resb 30

section .text
    _start:

    mov edx, msj
    call puts

    mov edx, cad1
    call capturar
    call buscar

    mov eax, 1
    mov ebx, 0
    int 80h
```

```
C/C++
buscar:
    pushad

    mov esi, 0
    mov ah, 0
```

```
call getche
call salto

.buscar:
    cmp byte[edx+esi], 0
    je .salirBuscar

    cmp byte[edx+esi], al
    jne .saltar

    mov ebx, esi
    add bl, '0'
    push edx
    mov edx, msj1
    mov [edx+33], bl
    call puts
    call salto
    pop edx
    inc ah

    .saltar:
    inc esi

jmp .buscar

.salirBuscar:
add ah, '0'
mov edx, msj2
mov [edx+21], ah

call puts
call salto

popad
ret
```

```
Texto: hola mundo
o
La letra se encontro en posicion 1
La letra se encontro en posicion 9
La letra se encontro 2 veces
```

```
Texto: hello world
l
La letra se encontro en posicion 2
La letra se encontro en posicion 3
La letra se encontro en posicion 9
La letra se encontro 3 veces
```

2. (25%) Implementar una subrutina donde se reciben dos cadenas, la primera está direccionada por edx y tiene la cadena de origen, la segunda está direccionada por edi y tiene la cadena de destino, se debe de tomar la cadena origen e invertirla y guardarla en la cadena destino, al final se debe mostrar en terminal.

```C/C++
section .data
    msj: db 'Texto: ',0x0

section .bss
    cad1: resb 30
    cad2: resb 30

section .text
    _start:

    mov edx, msj
    call puts

    mov edx, cad1
    call capturar
    mov edi, cad2
    call invertir

    mov edx, edi
    call puts
    call salto

    mov eax, 1
    mov ebx, 0
    int 80h
```

```C/C++
invertir:
    pushad
```

```
mov esi, 0

.conteo:
    cmp byte[edx+esi], 0
    je .salirConteo
    inc esi
jmp .conteo

.salirConteo:

mov ecx, 0

cmp esi, 0
je .salirInvertir

.invertir:
    dec esi

    mov al, byte[edx+esi]
    mov [edi+ecx], al

    inc ecx

    cmp esi, 0
    je .salirInvertir
jmp .invertir

.salirInvertir:
mov byte[edi+ecx], 0

popad
ret
```

```
Texto: Anitalavalatina
anitalavalatinA
```

```
Texto: espejo
ojepse
```

3. (25%) Implementar una subrutina que reciba una cadena de texto direccionada por edx y la pase a mayúsculas, al final se debe mostrar en terminal con new_puts.

```C/C++
section .data
    msj: db 'Texto: ',0x0

section .bss
    cad1: resb 30

section .text
    _start:

    mov edx, msj
    call puts

    mov edx, cad1
    call capturar
    call mayusculas

    call new_puts
    call salto

    mov eax, 1
    mov ebx, 0
    int 80h
```

```C/C++
mayusculas:
    pushad

    mov esi, 0

    .mayuscula:
        cmp byte[edx+esi], 0
        je .salirMayuscula

        sub byte[edx+esi], 32

        inc esi
    jmp .mayuscula

    .salirMayuscula:

    popad
    ret
```

```
C/C++
new_puts:
        pushad

        mov esi, 0

        .escribir:
        mov al, [edx+esi]
        cmp al, 0 ;CARACTER ESPECIAL DE TERMINACION
        je .salirEscribir
        inc esi
        call putchar
        jmp .escribir

        .salirEscribir:

        popad
        ret
```

```
Texto: minuscula
MINUSCULA
```

```
Texto: cadena
CADENA
```

4. (25%) Implementar una subrutina que reciba una cadena de texto direccionada por edx, debe contar cuantas vocales aparecen en la cadena, debe mostrar la cantidad individual y total de ellas.

```
C/C++
section .data
    msj: db 'Texto: ',0x0
    msj3: db 'Hay # @',0x0
    msj4: db 'Se encontro # vocales',0x0

section .bss
    cad1: resb 30

section .text
    _start:

    mov edx, msj
    call puts
```

```
    mov edx, cad1
    call capturar
    call vocales

    mov eax, 1
    mov ebx, 0
    int 80h
```

```C/C++
vocales:
    pushad

    mov eax, 0

    mov eax, 'u'
    push eax
    mov eax, 'o'
    push eax
    mov eax, 'i'
    push eax
    mov eax, 'e'
    push eax
    mov eax, 'a'
    push eax

    mov bh, 0 ;CONTADOR TOTAL DE VOCALES

    mov ecx, 5

    .vocal:
        mov esi, 0
        mov bl, 0 ;CONTADOR DE VOCALES INDIVIDUALES
        pop eax ;CARGA EL VOCAL EN AL

        .buscar:
            cmp byte[edx+esi], 0
            je .salirBuscar

            cmp byte[edx+esi], al
            jne .saltar

            inc bl
            inc bh

            .saltar:
            inc esi
```

```
        jmp .buscar

        .salirBuscar:
        push edx
        mov edx, msj3
        add bl, '0'
        mov [edx+4], bl
        mov [edx+6], al
        call puts
        call salto
        pop edx

    loop .vocal

    mov edx, msj4
    add bh, '0'
    mov [edx+12], bh
    call puts
    call salto

    popad
    ret
```

```
Texto: abecedario
Hay 2 a
Hay 2 e
Hay 1 i
Hay 1 o
Hay 0 u
Se encontro 6 vocales
```

```
Texto: metamorfosis
Hay 1 a
Hay 1 e
Hay 1 i
Hay 2 o
Hay 0 u
Se encontro 5 vocales
```

Dificultades

En el desarrollo de la primera subrutina, al momento de intentar modificar un caracter (8 bits) de una cadena en el segmento de memoria utilice el registro ebx (32 bits). Como el caracter modificado era el ultimo de la cadena, pareciera que la cadena se modificó sin problemas pero esto afectó a la cadena contigua.

Conclusiones

La implementación de las subrutinas en lenguaje ensamblador permitió comprender y aplicar los diversos modos de direccionamiento, como el indexado con desplazamiento (byte[edx+esi]) y el directo ([edx+33]), esenciales para la manipulación eficiente de cadenas de texto.

Se logró el objetivo de buscar, contar, invertir y transformar caracteres, demostrando la importancia de seleccionar el modo de direccionamiento adecuado.